

Python Fundamentals – Detailed Notes

1. Installing Visual Studio Code

Introduction to Visual Studio Code

Visual Studio Code (VS Code) is one of the most popular source-code editors developed by Microsoft. It is lightweight, fast, customizable, and supports multiple programming languages including Python, JavaScript, Java, C++, and many more.

VS Code is widely used because of:

- Fast performance
- Built-in terminal
- Extension support
- Syntax highlighting
- Debugging tools
- Git integration
- Intelligent code suggestions (IntelliSense)

Python developers commonly use VS Code because it provides excellent support for writing, debugging, and running Python programs.

System Requirements for VS Code

Before installing VS Code, ensure your system meets the minimum requirements:

Windows

- Windows 10 or later
- Minimum 1 GB RAM
- 200 MB disk space

macOS

- macOS 10.11 or later

Linux

- Ubuntu, Debian, Fedora, or other Linux distributions

Steps to Install Visual Studio Code

Step 1: Visit the Official Website

Open your browser and visit:

<https://code.visualstudio.com>

Step 2: Download VS Code

Click the download button according to your operating system:

- Windows
- macOS
- Linux

The installer file will start downloading.

Step 3: Run the Installer

Locate the downloaded installer file and double-click it.

For Windows:

- Accept the license agreement.
- Choose installation location.
- Click “Next”.

Important options to select:

- Add to PATH
- Register Code as editor
- Create desktop icon

These options improve usability.

Step 4: Complete Installation

Click “Install”.

After installation finishes:

- Click “Finish”
 - Launch VS Code
-

Installing Python Extension in VS Code

VS Code alone cannot fully support Python programming. You must install the Python extension.

Steps:

1. Open VS Code.
2. Click the Extensions icon from the left sidebar.
3. Search for “Python”.
4. Install the extension developed by Microsoft.

This extension provides:

- Syntax highlighting
 - Auto-completion
 - Debugging
 - Linting
 - Code formatting
 - Jupyter Notebook support
-

Features of VS Code for Python Development

1. IntelliSense

Provides smart suggestions while typing code.

2. Integrated Terminal

Allows running Python programs directly inside VS Code.

3. Debugging

Helps identify errors and track program execution.

4. Extensions

Supports thousands of extensions.

5. Git Integration

Supports version control systems.

Advantages of VS Code

- Free and open-source
- Cross-platform support

- Lightweight and fast
 - Highly customizable
 - Excellent developer community
-

2. Installing Python

Introduction to Python Installation

Python is a high-level, interpreted programming language known for its simplicity and readability. Before writing Python programs, Python must be installed on the computer.

Python can be downloaded from the official website:

<https://www.python.org>

Steps to Install Python on Windows

Step 1: Visit Python Official Website

Open:

<https://www.python.org/downloads/>

Step 2: Download Python

Click “Download Python”.

The latest stable version will start downloading.

Step 3: Run the Installer

Double-click the downloaded installer.

Important:

Check the option:

“Add Python to PATH”

This is extremely important because it allows Python to run from the command prompt.

Step 4: Install Python

Click “Install Now”.

Wait for installation to complete.

Step 5: Verify Installation

Open Command Prompt and type:

```
python --version
```

If installed correctly, Python version will appear.

Example:

```
Python 3.12.0
```

Installing Python on macOS

Steps:

1. Download Python installer from python.org.
2. Open the installer package.
3. Follow installation instructions.
4. Verify using terminal:

```
python3 --version
```

Installing Python on Linux

Most Linux systems already include Python.

To verify:

```
python3 --version
```

If not installed:

Ubuntu/Debian

```
sudo apt update
```

```
sudo apt install python3
```

Running the First Python Program

Method 1: Using Python Interpreter

Open terminal and type:

```
python
```

Now type:

```
print("Hello World")
```

Method 2: Using VS Code

1. Create a file named hello.py
2. Write:

```
print("Hello World")
```

3. Run the file.

Output:

Hello World

Why Python is Popular

Python is widely used because:

- Easy syntax
 - Large community
 - Huge libraries
 - Cross-platform support
 - Used in AI and Data Science
 - Rapid development
-

Applications of Python

Python is used in:

- Web development
- Artificial Intelligence
- Machine Learning
- Data Science
- Automation
- Cybersecurity
- Game development

- Desktop applications
-

3. Introduction to Python

What is Python?

Python is a general-purpose, high-level programming language created by Guido van Rossum and released in 1991.

Python focuses on:

- Readability
- Simplicity
- Productivity

Python syntax is easy to understand compared to many other programming languages.

Features of Python

1. Easy to Learn

Python syntax resembles English language.

2. Interpreted Language

Programs are executed line by line.

3. Object-Oriented

Supports classes and objects.

4. Platform Independent

Python code works on Windows, Linux, and macOS.

5. Large Standard Library

Provides built-in modules and functions.

6. Open Source

Free to use and modify.

Basic Python Syntax

Example:

```
print("Welcome to Python")
```

Output:

Welcome to Python

Python Comments

Comments are ignored by the interpreter.

Single-line Comment

```
# This is a comment
```

Multi-line Comment

```
"""
```

This is a

multi-line comment

```
"""
```

Input and Output in Python

Output

```
print("Hello")
```

Input

```
name = input("Enter your name: ")
```

```
print(name)
```

Data Types in Python

Python supports multiple data types.

Numeric Types

- int
- float
- complex

Sequence Types

- list
- tuple
- string

Boolean Type

- True
- False

Mapping Type

- dictionary
-

Python Operators

Arithmetic Operators

+

-

*

/

%

Comparison Operators

==

!=

>

<

Logical Operators

and

or

not

Advantages of Python

- Beginner friendly
- Huge community support
- Fast development
- Less code compared to other languages
- Excellent for automation

Limitations of Python

- Slower than compiled languages
 - High memory consumption
 - Not ideal for mobile development
-

4. Variables in Python

Introduction to Variables

Variables are containers used to store data values.

In Python, variables are created automatically when a value is assigned.

Example:

```
name = "Shubham"
```

```
age = 22
```

Here:

- name stores a string
 - age stores an integer
-

Characteristics of Variables

- Store data
 - Reusable
 - Changeable during program execution
 - Improve code readability
-

Creating Variables in Python

Example:

```
x = 10
```

```
y = 20
```

```
sum = x + y
```

```
print(sum)
```

Output:

30

Dynamic Typing in Python

Python is dynamically typed.

This means variable type is decided automatically.

Example:

```
x = 10
```

```
x = "Hello"
```

Python allows changing data type during execution.

Multiple Variable Assignment

Example:

```
a, b, c = 1, 2, 3
```

Assigning Same Value to Multiple Variables

```
x = y = z = 100
```

Types of Variables

1. Local Variables

Declared inside a function.

2. Global Variables

Declared outside functions.

Variable Naming Examples

Valid Names

name

student_name

age1

`_total`

Invalid Names

`1name`

`student-name`

`class`

Constants in Python

Python does not provide true constants.

Convention:

`PI = 3.14`

Uppercase names are treated as constants.

Memory Allocation in Variables

Python automatically allocates memory when variables are created.

Developers do not need manual memory management.

Advantages of Variables

- Store reusable data
 - Reduce repetition
 - Improve readability
 - Simplify calculations
-

5. Checking the Type of Variables

Introduction

Python provides the `type()` function to identify the data type of a variable.

Syntax

`type(variable_name)`

Checking Integer Type

```
x = 10
```

```
print(type(x))
```

Output:

```
<class 'int'>
```

Checking Float Type

```
x = 10.5
```

```
print(type(x))
```

Output:

```
<class 'float'>
```

Checking String Type

```
name = "Python"
```

```
print(type(name))
```

Output:

```
<class 'str'>
```

Checking Boolean Type

```
status = True
```

```
print(type(status))
```

Output:

```
<class 'bool'>
```

Checking List Type

```
numbers = [1, 2, 3]
```

```
print(type(numbers))
```

Output:

```
<class 'list'>
```

Checking Tuple Type

```
data = (1, 2, 3)
print(type(data))
```

Output:

```
<class 'tuple'>
```

Checking Dictionary Type

```
student = {"name": "Ram", "age": 20}
print(type(student))
```

Output:

```
<class 'dict'>
```

Importance of Checking Data Types

- Prevents runtime errors
 - Helps debugging
 - Ensures correct operations
 - Improves code quality
-

Type Conversion

Python allows conversion between data types.

Example:

```
x = "10"
y = int(x)
print(type(y))
```

Output:

```
<class 'int'>
```

6. Rules for Variables Declaration

Introduction

Python follows specific naming rules while declaring variables.

Incorrect variable names produce syntax errors.

Rules for Variable Names

Rule 1: Variable Name Must Start with Letter or Underscore

Valid

name

_age

Invalid

1name

Rule 2: Variable Name Cannot Contain Spaces

Invalid

student name = "Ram"

Correct

student_name = "Ram"

Rule 3: Special Characters Are Not Allowed

Invalid

name@

student-name

Only underscore _ is allowed.

Rule 4: Python Keywords Cannot Be Used

Invalid

class = 10

Keywords are reserved words.

Rule 5: Variable Names Are Case Sensitive

age = 20

Age = 30

These are treated as different variables.

Rule 6: Use Meaningful Names

Poor Naming

x = 100

Better Naming

student_marks = 100

Meaningful names improve readability.

Naming Conventions

Snake Case

Recommended in Python.

student_name

employee_salary

Best Practices

- Use meaningful names
 - Avoid very long names
 - Follow naming conventions
 - Keep names readable
 - Avoid unnecessary abbreviations
-

Common Mistakes

- Using keywords
- Starting with numbers
- Using spaces

- Using special symbols
-

7. Keywords in Python

Introduction to Keywords

Keywords are reserved words in Python with predefined meanings.

These words cannot be used as variable names, function names, or identifiers.

Characteristics of Keywords

- Reserved by Python
 - Have special meaning
 - Cannot be redefined
 - Case sensitive
-

Checking Python Keywords

Python provides a module called keyword.

Example:

```
import keyword  
print(keyword.kwlist)
```

This displays all Python keywords.

Common Python Keywords

Keyword Purpose

if	Conditional statement
else	Alternative condition
elif	Multiple conditions
for	Looping
while	Repeated execution
break	Exit loop

Keyword Purpose

continue Skip iteration

def Function definition

class Class creation

return Return value

import Import module

try Exception handling

except Handle errors

True Boolean true

False Boolean false

None Represents null value

Examples of Keywords

Using if Statement

```
age = 18
```

```
if age >= 18:
```

```
    print("Eligible")
```

```
else:
```

```
    print("Not Eligible")
```

Loop Keywords Example

```
for i in range(5):
```

```
    print(i)
```

Function Keyword Example

```
def greet():
```

```
    print("Hello")
```

Exception Handling Keywords

try:

```
x = 10 / 0
```

except:

```
print("Error occurred")
```

Why Keywords Are Important

- Define language structure
 - Control program flow
 - Enable object-oriented programming
 - Support exception handling
 - Improve readability
-

Conclusion

Python is one of the most powerful and beginner-friendly programming languages in the world. Understanding the installation process, variables, data types, naming rules, and keywords forms the foundation of Python programming.

Visual Studio Code provides an efficient environment for development, while Python's simple syntax makes it easy to learn and use. Mastering these basics is essential before moving toward advanced topics such as functions, object-oriented programming, file handling, web development, data science, and machine learning.

Strong fundamentals lead to better programming skills, cleaner code, and improved problem-solving ability.